

## 9 | Modern Recurrent Neural Networks

We have introduced the basics of RNNs, which can better handle sequence data. For demonstration, we implemented RNN-based language models on text data. However, such techniques may not be sufficient for practitioners when they face a wide range of sequence learning problems nowadays.

For instance, a notable issue in practice is the numerical instability of RNNs. Although we have applied implementation tricks such as gradient clipping, this issue can be alleviated further with more sophisticated designs of sequence models. Specifically, gated RNNs are much more common in practice. We will begin by introducing two of such widely-used networks, namely *gated recurrent units* (GRUs) and *long short-term memory* (LSTM). Furthermore, we will expand the RNN architecture with a single unidirectional hidden layer that has been discussed so far. We will describe deep architectures with multiple hidden layers, and discuss the bidirectional design with both forward and backward recurrent computations. Such expansions are frequently adopted in modern recurrent networks. When explaining these RNN variants, we continue to consider the same language modeling problem introduced in [Chapter 8](#).

In fact, language modeling reveals only a small fraction of what sequence learning is capable of. In a variety of sequence learning problems, such as automatic speech recognition, text to speech, and machine translation, both inputs and outputs are sequences of arbitrary length. To explain how to fit this type of data, we will take machine translation as an example, and introduce the encoder-decoder architecture based on RNNs and beam search for sequence generation.

### 9.1 Gated Recurrent Units (GRU)

In [Section 8.7](#), we discussed how gradients are calculated in RNNs. In particular we found that long products of matrices can lead to vanishing or exploding gradients. Let us briefly think about what such gradient anomalies mean in practice:

- We might encounter a situation where an early observation is highly significant for predicting all future observations. Consider the somewhat contrived case where the first observation contains a checksum and the goal is to discern whether the checksum is correct at the end of the sequence. In this case, the influence of the first token is vital. We would like to have some mechanisms for storing vital early information in a *memory cell*. Without such a mechanism, we will have to assign a very large gradient to this observation, since it affects all the subsequent observations.
- We might encounter situations where some tokens carry no pertinent observation. For instance, when parsing a web page there might be auxiliary HTML code that is irrelevant for the purpose of assessing the sentiment conveyed on the page. We would like to have some mechanism for *skipping* such tokens in the latent state representation.

- We might encounter situations where there is a logical break between parts of a sequence. For instance, there might be a transition between chapters in a book, or a transition between a bear and a bull market for securities. In this case it would be nice to have a means of *resetting* our internal state representation.

A number of methods have been proposed to address this. One of the earliest is long short-term memory (Hochreiter & Schmidhuber, 1997) which we will discuss in [Section 9.2](#). The gated recurrent unit (GRU) (Cho et al., 2014a) is a slightly more streamlined variant that often offers comparable performance and is significantly faster to compute (Chung et al., 2014). Due to its simplicity, let us start with the GRU.

### 9.1.1 Gated Hidden State

The key distinction between vanilla RNNs and GRUs is that the latter support gating of the hidden state. This means that we have dedicated mechanisms for when a hidden state should be *updated* and also when it should be *reset*. These mechanisms are learned and they address the concerns listed above. For instance, if the first token is of great importance we will learn not to update the hidden state after the first observation. Likewise, we will learn to skip irrelevant temporary observations. Last, we will learn to reset the latent state whenever needed. We discuss this in detail below.

#### Reset Gate and Update Gate

The first thing we need to introduce are the *reset gate* and the *update gate*. We engineer them to be vectors with entries in  $(0, 1)$  such that we can perform convex combinations. For instance, a reset gate would allow us to control how much of the previous state we might still want to remember. Likewise, an update gate would allow us to control how much of the new state is just a copy of the old state.

We begin by engineering these gates. [Fig. 9.1.1](#) illustrates the inputs for both the reset and update gates in a GRU, given the input of the current time step and the hidden state of the previous time step. The outputs of two gates are given by two fully-connected layers with a sigmoid activation function.

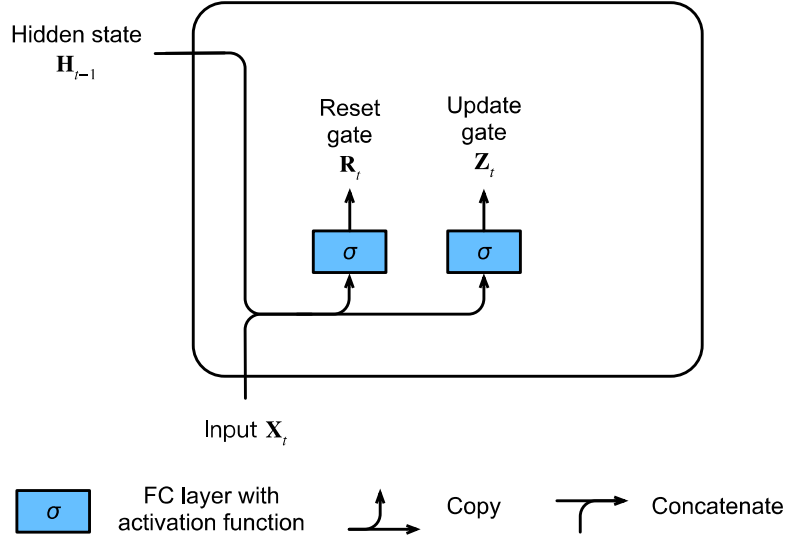


Fig. 9.1.1: Computing the reset gate and the update gate in a GRU model.

Mathematically, for a given time step  $t$ , suppose that the input is a minibatch  $\mathbf{X}_t \in \mathbb{R}^{n \times d}$  (number of examples:  $n$ , number of inputs:  $d$ ) and the hidden state of the previous time step is  $\mathbf{H}_{t-1} \in \mathbb{R}^{n \times h}$  (number of hidden units:  $h$ ). Then, the reset gate  $\mathbf{R}_t \in \mathbb{R}^{n \times h}$  and update gate  $\mathbf{Z}_t \in \mathbb{R}^{n \times h}$  are computed as follows:

$$\begin{aligned}\mathbf{R}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xr} + \mathbf{H}_{t-1} \mathbf{W}_{hr} + \mathbf{b}_r), \\ \mathbf{Z}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xz} + \mathbf{H}_{t-1} \mathbf{W}_{hz} + \mathbf{b}_z),\end{aligned}\tag{9.1.1}$$

where  $\mathbf{W}_{xr}, \mathbf{W}_{xz} \in \mathbb{R}^{d \times h}$  and  $\mathbf{W}_{hr}, \mathbf{W}_{hz} \in \mathbb{R}^{h \times h}$  are weight parameters and  $\mathbf{b}_r, \mathbf{b}_z \in \mathbb{R}^{1 \times h}$  are biases. Note that broadcasting (see Section 2.1.3) is triggered during the summation. We use sigmoid functions (as introduced in Section 4.1) to transform input values to the interval  $(0, 1)$ .

### Candidate Hidden State

Next, let us integrate the reset gate  $\mathbf{R}_t$  with the regular latent state updating mechanism in (8.4.5). It leads to the following *candidate hidden state*  $\tilde{\mathbf{H}}_t \in \mathbb{R}^{n \times h}$  at time step  $t$ :

$$\tilde{\mathbf{H}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xh} + (\mathbf{R}_t \odot \mathbf{H}_{t-1}) \mathbf{W}_{hh} + \mathbf{b}_h),\tag{9.1.2}$$

where  $\mathbf{W}_{xh} \in \mathbb{R}^{d \times h}$  and  $\mathbf{W}_{hh} \in \mathbb{R}^{h \times h}$  are weight parameters,  $\mathbf{b}_h \in \mathbb{R}^{1 \times h}$  is the bias, and the symbol  $\odot$  is the Hadamard (elementwise) product operator. Here we use a nonlinearity in the form of  $\tanh$  to ensure that the values in the candidate hidden state remain in the interval  $(-1, 1)$ .

The result is a *candidate* since we still need to incorporate the action of the update gate. Comparing with (8.4.5), now the influence of the previous states can be reduced with the elementwise multiplication of  $\mathbf{R}_t$  and  $\mathbf{H}_{t-1}$  in (9.1.2). Whenever the entries in the reset gate  $\mathbf{R}_t$  are close to 1, we recover a vanilla RNN such as in (8.4.5). For all entries of the reset gate  $\mathbf{R}_t$  that are close to 0, the candidate hidden state is the result of an MLP with  $\mathbf{X}_t$  as the input. Any pre-existing hidden state is thus *reset* to defaults.

Fig. 9.1.2 illustrates the computational flow after applying the reset gate.

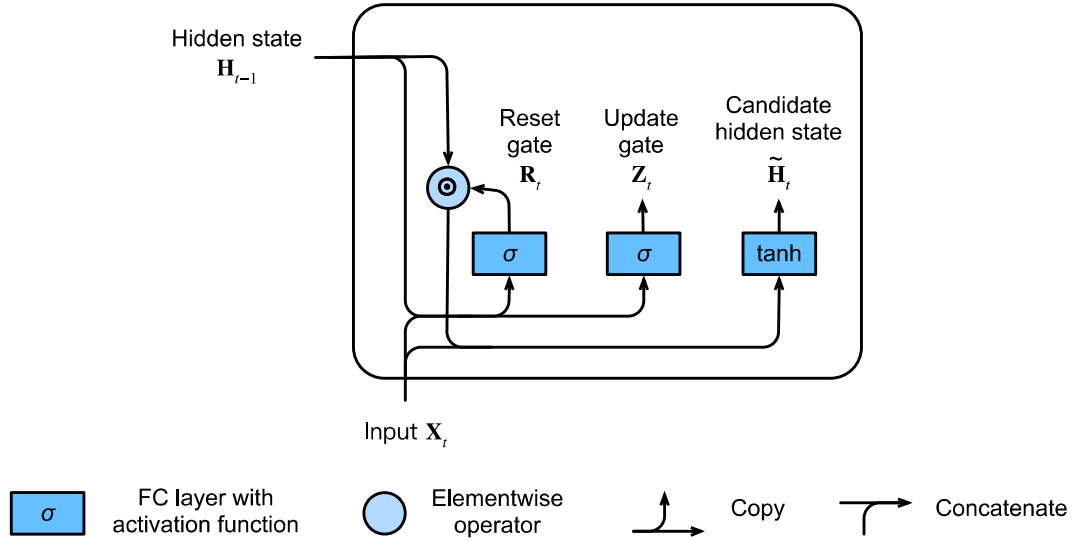


Fig. 9.1.2: Computing the candidate hidden state in a GRU model.

### Hidden State

Finally, we need to incorporate the effect of the update gate  $\mathbf{Z}_t$ . This determines the extent to which the new hidden state  $\mathbf{H}_t \in \mathbb{R}^{n \times h}$  is just the old state  $\mathbf{H}_{t-1}$  and by how much the new candidate state  $\tilde{\mathbf{H}}_t$  is used. The update gate  $\mathbf{Z}_t$  can be used for this purpose, simply by taking elementwise convex combinations between both  $\mathbf{H}_{t-1}$  and  $\tilde{\mathbf{H}}_t$ . This leads to the final update equation for the GRU:

$$\mathbf{H}_t = \mathbf{Z}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{Z}_t) \odot \tilde{\mathbf{H}}_t. \quad (9.1.3)$$

Whenever the update gate  $\mathbf{Z}_t$  is close to 1, we simply retain the old state. In this case the information from  $\mathbf{X}_t$  is essentially ignored, effectively skipping time step  $t$  in the dependency chain. In contrast, whenever  $\mathbf{Z}_t$  is close to 0, the new latent state  $\mathbf{H}_t$  approaches the candidate latent state  $\tilde{\mathbf{H}}_t$ . These designs can help us cope with the vanishing gradient problem in RNNs and better capture dependencies for sequences with large time step distances. For instance, if the update gate has been close to 1 for all the time steps of an entire subsequence, the old hidden state at the time step of its beginning will be easily retained and passed to its end, regardless of the length of the subsequence.

Fig. 9.1.3 illustrates the computational flow after the update gate is in action.

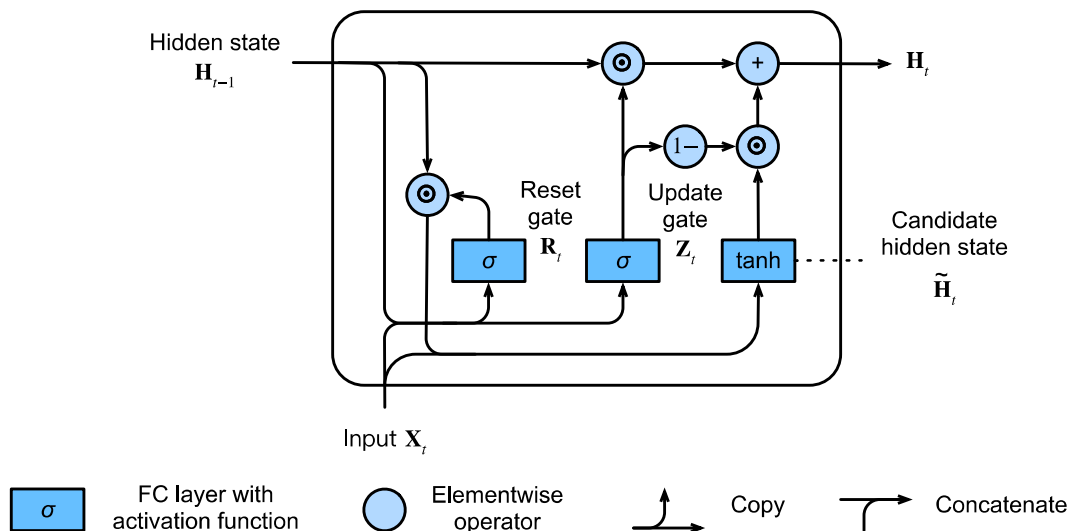


Fig. 9.1.3: Computing the hidden state in a GRU model.

In summary, GRUs have the following two distinguishing features:

- Reset gates help capture short-term dependencies in sequences.
- Update gates help capture long-term dependencies in sequences.

### 9.1.2 Implementation from Scratch

To gain a better understanding of the GRU model, let us implement it from scratch. We begin by reading the time machine dataset that we used in [Section 8.5](#). The code for reading the dataset is given below.

```
import torch
from torch import nn
from d2l import torch as d2l

batch_size, num_steps = 32, 35
train_iter, vocab = d2l.load_data_time_machine(batch_size, num_steps)
```

#### Initializing Model Parameters

The next step is to initialize the model parameters. We draw the weights from a Gaussian distribution with standard deviation to be 0.01 and set the bias to 0. The hyperparameter `num_hiddens` defines the number of hidden units. We instantiate all weights and biases relating to the update gate, the reset gate, the candidate hidden state, and the output layer.

```
def get_params(vocab_size, num_hiddens, device):
    num_inputs = num_outputs = vocab_size

    def normal(shape):
        return torch.randn(size=shape, device=device)*0.01
```

(continues on next page)

```

def three():
    return (normal((num_inputs, num_hiddens)),
            normal((num_hiddens, num_hiddens)),
            torch.zeros(num_hiddens, device=device))

W_xz, W_hz, b_z = three() # Update gate parameters
W_xr, W_hr, b_r = three() # Reset gate parameters
W_xh, W_hh, b_h = three() # Candidate hidden state parameters
# Output layer parameters
W_hq = normal((num_hiddens, num_outputs))
b_q = torch.zeros(num_outputs, device=device)
# Attach gradients
params = [W_xz, W_hz, b_z, W_xr, W_hr, b_r, W_xh, W_hh, b_h, W_hq, b_q]
for param in params:
    param.requires_grad_(True)
return params

```

## Defining the Model

Now we will define the hidden state initialization function `init_gru_state`. Just like the `init_rnn_state` function defined in [Section 8.5](#), this function returns a tensor with a shape (batch size, number of hidden units) whose values are all zeros.

```

def init_gru_state(batch_size, num_hiddens, device):
    return (torch.zeros((batch_size, num_hiddens), device=device), )

```

Now we are ready to define the GRU model. Its structure is the same as that of the basic RNN cell, except that the update equations are more complex.

```

def gru(inputs, state, params):
    W_xz, W_hz, b_z, W_xr, W_hr, b_r, W_xh, W_hh, b_h, W_hq, b_q = params
    H, = state
    outputs = []
    for X in inputs:
        Z = torch.sigmoid((X @ W_xz) + (H @ W_hz) + b_z)
        R = torch.sigmoid((X @ W_xr) + (H @ W_hr) + b_r)
        H_tilda = torch.tanh((X @ W_xh) + ((R * H) @ W_hh) + b_h)
        H = Z * H + (1 - Z) * H_tilda
        Y = H @ W_hq + b_q
        outputs.append(Y)
    return torch.cat(outputs, dim=0), (H,)

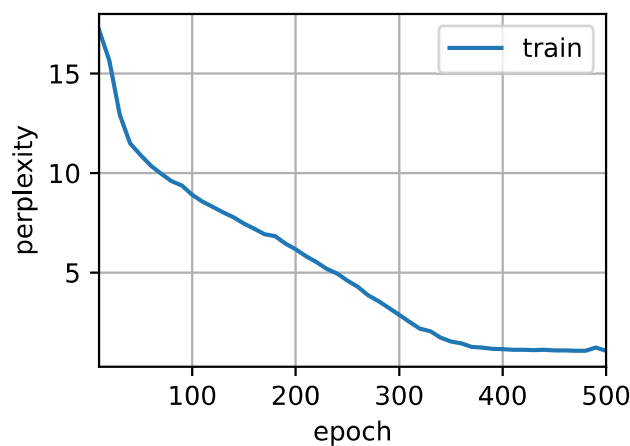
```

## Training and Predicting

Training and prediction work in exactly the same manner as in [Section 8.5](#). After training, we print out the perplexity on the training set and the predicted sequence following the provided prefixes “time traveller” and “traveller”, respectively.

```
vocab_size, num_hiddens, device = len(vocab), 256, d2l.try_gpu()
num_epochs, lr = 500, 1
model = d2l.RNNModelScratch(len(vocab), num_hiddens, device, get_params,
                             init_gru_state, gru)
d2l.train_ch8(model, train_iter, vocab, lr, num_epochs, device)
```

```
perplexity 1.1, 26805.4 tokens/sec on cuda:0
time traveller you can show black is white by argument said filby
travelleryou can show black is white by argument said filby
```

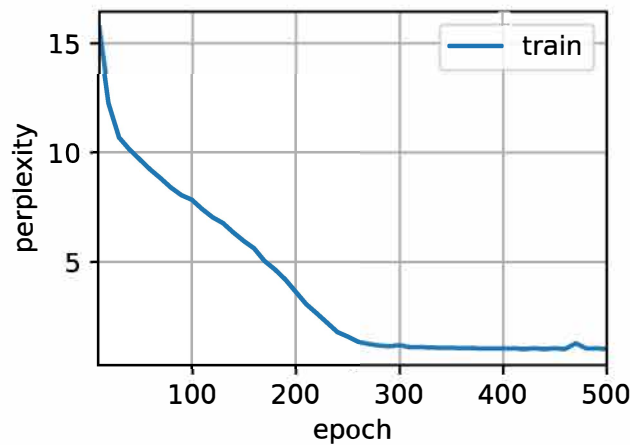


### 9.1.3 Concise Implementation

In high-level APIs, we can directly instantiate a GPU model. This encapsulates all the configuration detail that we made explicit above. The code is significantly faster as it uses compiled operators rather than Python for many details that we spelled out before.

```
num_inputs = vocab_size
gru_layer = nn.GRU(num_inputs, num_hiddens)
model = d2l.RNNModel(gru_layer, len(vocab))
model = model.to(device)
d2l.train_ch8(model, train_iter, vocab, lr, num_epochs, device)
```

```
perplexity 1.0, 334686.7 tokens/sec on cuda:0
time traveller for so it will be convenient to speak of himwas e
travelleryou can show black is white by argument said filby
```



## Summary

- Gated RNNs can better capture dependencies for sequences with large time step distances.
- Reset gates help capture short-term dependencies in sequences.
- Update gates help capture long-term dependencies in sequences.
- GRUs contain basic RNNs as their extreme case whenever the reset gate is switched on. They can also skip subsequences by turning on the update gate.

## Exercises

1. Assume that we only want to use the input at time step  $t'$  to predict the output at time step  $t > t'$ . What are the best values for the reset and update gates for each time step?
2. Adjust the hyperparameters and analyze the their influence on running time, perplexity, and the output sequence.
3. Compare runtime, perplexity, and the output strings for `rnn.RNN` and `rnn.GRU` implementations with each other.
4. What happens if you implement only parts of a GRU, e.g., with only a reset gate or only an update gate?

## 9.2 Long Short-Term Memory (LSTM)

The challenge to address long-term information preservation and short-term input skipping in latent variable models has existed for a long time. One of the earliest approaches to address this was the long short-term memory (LSTM) (Hochreiter & Schmidhuber, 1997). It shares many of the properties of the GRU. Interestingly, LSTMs have a slightly more complex design than GRUs but predates GRUs by almost two decades.

### 9.2.1 Gated Memory Cell

Arguably LSTM's design is inspired by logic gates of a computer. LSTM introduces a *memory cell* (or *cell* for short) that has the same shape as the hidden state (some literatures consider the memory cell as a special type of the hidden state), engineered to record additional information. To control the memory cell we need a number of gates. One gate is needed to read out the entries from the cell. We will refer to this as the *output gate*. A second gate is needed to decide when to read data into the cell. We refer to this as the *input gate*. Last, we need a mechanism to reset the content of the cell, governed by a *forget gate*. The motivation for such a design is the same as that of GRUs, namely to be able to decide when to remember and when to ignore inputs in the hidden state via a dedicated mechanism. Let us see how this works in practice.

#### Input Gate, Forget Gate, and Output Gate

Just like in GRUs, the data feeding into the LSTM gates are the input at the current time step and the hidden state of the previous time step, as illustrated in Fig. 9.2.1. They are processed by three fully-connected layers with a sigmoid activation function to compute the values of the input, forget, and output gates. As a result, values of the three gates are in the range of (0, 1).

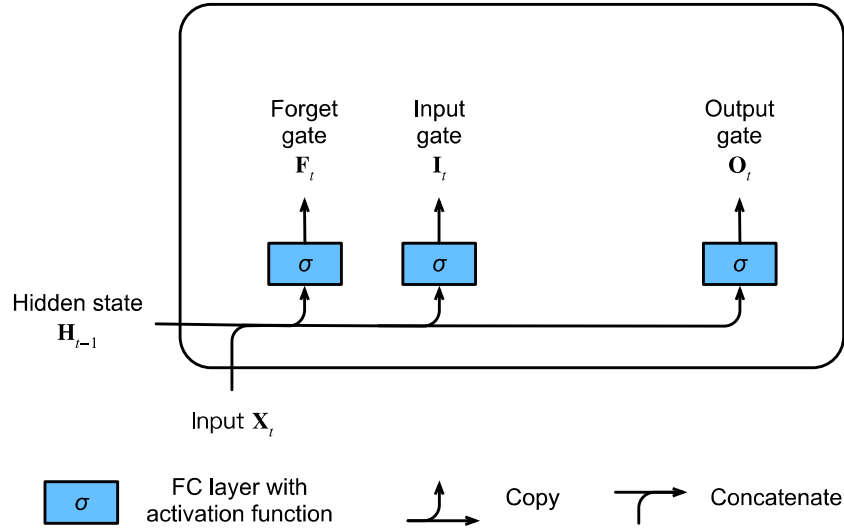


Fig. 9.2.1: Computing the input gate, the forget gate, and the output gate in an LSTM model.

Mathematically, suppose that there are  $h$  hidden units, the batch size is  $n$ , and the number of inputs is  $d$ . Thus, the input is  $\mathbf{X}_t \in \mathbb{R}^{n \times d}$  and the hidden state of the previous time step is  $\mathbf{H}_{t-1} \in \mathbb{R}^{n \times h}$ . Correspondingly, the gates at time step  $t$  are defined as follows: the input gate is  $\mathbf{I}_t \in \mathbb{R}^{n \times h}$ , the forget gate is  $\mathbf{F}_t \in \mathbb{R}^{n \times h}$ , and the output gate is  $\mathbf{O}_t \in \mathbb{R}^{n \times h}$ . They are calculated as follows:

$$\begin{aligned}\mathbf{I}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xi} + \mathbf{H}_{t-1} \mathbf{W}_{hi} + \mathbf{b}_i), \\ \mathbf{F}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xf} + \mathbf{H}_{t-1} \mathbf{W}_{hf} + \mathbf{b}_f), \\ \mathbf{O}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xo} + \mathbf{H}_{t-1} \mathbf{W}_{ho} + \mathbf{b}_o),\end{aligned}\tag{9.2.1}$$

where  $\mathbf{W}_{xi}, \mathbf{W}_{xf}, \mathbf{W}_{xo} \in \mathbb{R}^{d \times h}$  and  $\mathbf{W}_{hi}, \mathbf{W}_{hf}, \mathbf{W}_{ho} \in \mathbb{R}^{h \times h}$  are weight parameters and  $\mathbf{b}_i, \mathbf{b}_f, \mathbf{b}_o \in \mathbb{R}^{1 \times h}$  are bias parameters.

## Candidate Memory Cell

Next we design the memory cell. Since we have not specified the action of the various gates yet, we first introduce the *candidate* memory cell  $\tilde{\mathbf{C}}_t \in \mathbb{R}^{n \times h}$ . Its computation is similar to that of the three gates described above, but using a tanh function with a value range for  $(-1, 1)$  as the activation function. This leads to the following equation at time step  $t$ :

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xc} + \mathbf{H}_{t-1} \mathbf{W}_{hc} + \mathbf{b}_c), \quad (9.2.2)$$

where  $\mathbf{W}_{xc} \in \mathbb{R}^{d \times h}$  and  $\mathbf{W}_{hc} \in \mathbb{R}^{h \times h}$  are weight parameters and  $\mathbf{b}_c \in \mathbb{R}^{1 \times h}$  is a bias parameter.

A quick illustration of the candidate memory cell is shown in Fig. 9.2.2.

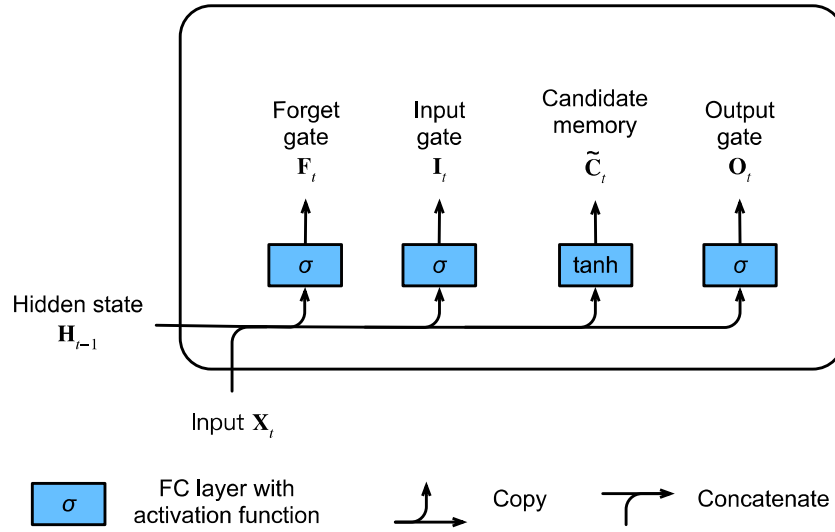


Fig. 9.2.2: Computing the candidate memory cell in an LSTM model.

## Memory Cell

In GRUs, we have a mechanism to govern input and forgetting (or skipping). Similarly, in LSTMs we have two dedicated gates for such purposes: the input gate  $I_t$  governs how much we take new data into account via  $\tilde{C}_t$  and the forget gate  $F_t$  addresses how much of the old memory cell content  $C_{t-1} \in \mathbb{R}^{n \times h}$  we retain. Using the same pointwise multiplication trick as before, we arrive at the following update equation:

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t. \quad (9.2.3)$$

If the forget gate is always approximately 1 and the input gate is always approximately 0, the past memory cells  $C_{t-1}$  will be saved over time and passed to the current time step. This design is introduced to alleviate the vanishing gradient problem and to better capture long range dependencies within sequences.

We thus arrive at the flow diagram in Fig. 9.2.3.

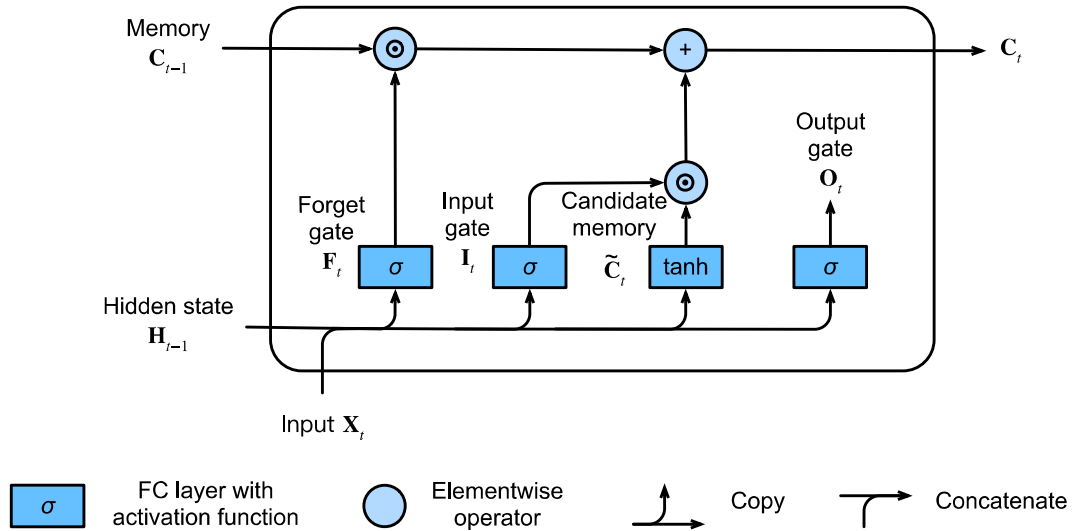


Fig. 9.2.3: Computing the memory cell in an LSTM model.

### Hidden State

Last, we need to define how to compute the hidden state  $\mathbf{H}_t \in \mathbb{R}^{n \times h}$ . This is where the output gate comes into play. In LSTM it is simply a gated version of the tanh of the memory cell. This ensures that the values of  $\mathbf{H}_t$  are always in the interval  $(-1, 1)$ .

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t). \quad (9.2.4)$$

Whenever the output gate approximates 1 we effectively pass all memory information through to the predictor, whereas for the output gate close to 0 we retain all the information only within the memory cell and perform no further processing.

Fig. 9.2.4 has a graphical illustration of the data flow.

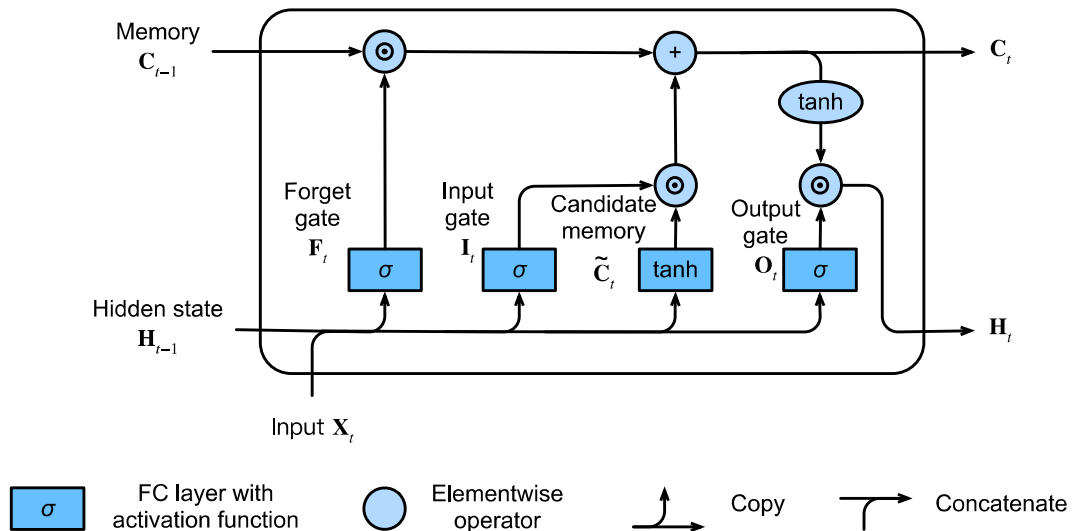


Fig. 9.2.4: Computing the hidden state in an LSTM model.

## 9.2.2 Implementation from Scratch

Now let us implement an LSTM from scratch. As same as the experiments in [Section 8.5](#), we first load the time machine dataset.

```
import torch
from torch import nn
from d2l import torch as d2l

batch_size, num_steps = 32, 35
train_iter, vocab = d2l.load_data_time_machine(batch_size, num_steps)
```

### Initializing Model Parameters

Next we need to define and initialize the model parameters. As previously, the hyperparameter `num_hiddens` defines the number of hidden units. We initialize weights following a Gaussian distribution with 0.01 standard deviation, and we set the biases to 0.

```
def get_lstm_params(vocab_size, num_hiddens, device):
    num_inputs = num_outputs = vocab_size

    def normal(shape):
        return torch.randn(size=shape, device=device)*0.01

    def three():
        return (normal((num_inputs, num_hiddens)),
                normal((num_hiddens, num_hiddens)),
                torch.zeros(num_hiddens, device=device))

    W_xi, W_hi, b_i = three() # Input gate parameters
    W_xf, W_hf, b_f = three() # Forget gate parameters
    W_xo, W_ho, b_o = three() # Output gate parameters
    W_xc, W_hc, b_c = three() # Candidate memory cell parameters
    # Output layer parameters
    W_hq = normal((num_hiddens, num_outputs))
    b_q = torch.zeros(num_outputs, device=device)
    # Attach gradients
    params = [W_xi, W_hi, b_i, W_xf, W_hf, b_f, W_xo, W_ho, b_o, W_xc, W_hc,
              b_c, W_hq, b_q]
    for param in params:
        param.requires_grad_(True)
    return params
```

## Defining the Model

In the initialization function, the hidden state of the LSTM needs to return an *additional* memory cell with a value of 0 and a shape of (batch size, number of hidden units). Hence we get the following state initialization.

```
def init_lstm_state(batch_size, num_hiddens, device):
    return (torch.zeros((batch_size, num_hiddens), device=device),
            torch.zeros((batch_size, num_hiddens), device=device))
```

The actual model is defined just like what we discussed before: providing three gates and an auxiliary memory cell. Note that only the hidden state is passed to the output layer. The memory cell  $C_t$  does not directly participate in the output computation.

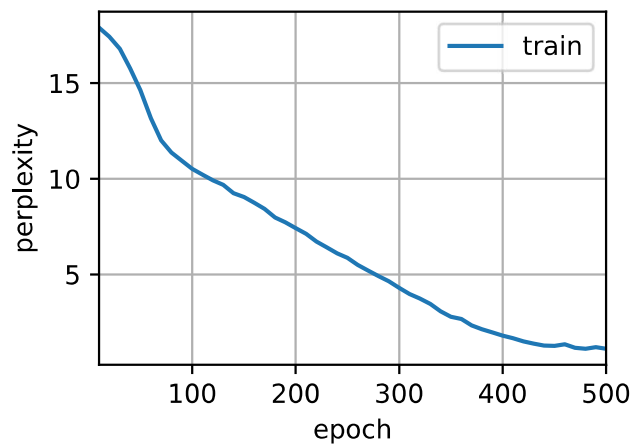
```
def lstm(inputs, state, params):
    [W_xi, W_hi, b_i, W_xf, W_hf, b_f, W_xo, W_ho, b_o, W_xc, W_hc, b_c,
     W_hq, b_q] = params
    (H, C) = state
    outputs = []
    for X in inputs:
        I = torch.sigmoid((X @ W_xi) + (H @ W_hi) + b_i)
        F = torch.sigmoid((X @ W_xf) + (H @ W_hf) + b_f)
        O = torch.sigmoid((X @ W_xo) + (H @ W_ho) + b_o)
        C_tilda = torch.tanh((X @ W_xc) + (H @ W_hc) + b_c)
        C = F * C + I * C_tilda
        H = O * torch.tanh(C)
        Y = (H @ W_hq) + b_q
        outputs.append(Y)
    return torch.cat(outputs, dim=0), (H, C)
```

## Training and Prediction

Let us train an LSTM as same as what we did in [Section 9.1](#), by instantiating the `RNNModelScratch` class as introduced in [Section 8.5](#).

```
vocab_size, num_hiddens, device = len(vocab), 256, d2l.try_gpu()
num_epochs, lr = 500, 1
model = d2l.RNNModelScratch(len(vocab), num_hiddens, device, get_lstm_params,
                             init_lstm_state, lstm)
d2l.train_ch8(model, train_iter, vocab, lr, num_epochs, device)
```

```
perplexity 1.1, 23381.5 tokens/sec on cuda:0
time traveller for so it will be convenient to speak of himwas e
traveller for so it will be convenient to speak of himwas e
```

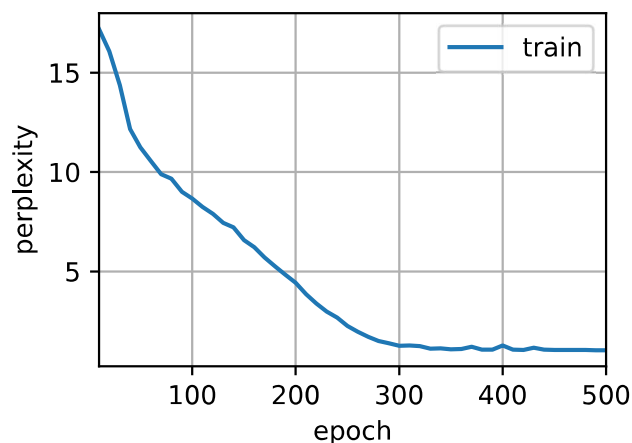


### 9.2.3 Concise Implementation

Using high-level APIs, we can directly instantiate an LSTM model. This encapsulates all the configuration details that we made explicit above. The code is significantly faster as it uses compiled operators rather than Python for many details that we spelled out in detail before.

```
num_inputs = vocab_size
lstm_layer = nn.LSTM(num_inputs, num_hiddens)
model = d2l.RNNModel(lstm_layer, len(vocab))
model = model.to(device)
d2l.train_ch8(model, train_iter, vocab, lr, num_epochs, device)
```

```
perplexity 1.0, 334397.8 tokens/sec on cuda:0
time traveller for so it will be convenient to speak of himwas e
traveller with a slight accession ofcheerfulness really thi
```



LSTMs are the prototypical latent variable autoregressive model with nontrivial state control. Many variants thereof have been proposed over the years, e.g., multiple layers, residual connections, different types of regularization. However, training LSTMs and other sequence models (such as GRUs) are quite costly due to the long range dependency of the sequence. Later we will encounter alternative models such as transformers that can be used in some cases.

## Summary

- LSTMs have three types of gates: input gates, forget gates, and output gates that control the flow of information.
- The hidden layer output of LSTM includes the hidden state and the memory cell. Only the hidden state is passed into the output layer. The memory cell is entirely internal.
- LSTMs can alleviate vanishing and exploding gradients.

## Exercises

1. Adjust the hyperparameters and analyze their influence on running time, perplexity, and the output sequence.
2. How would you need to change the model to generate proper words as opposed to sequences of characters?
3. Compare the computational cost for GRUs, LSTMs, and regular RNNs for a given hidden dimension. Pay special attention to the training and inference cost.
4. Since the candidate memory cell ensures that the value range is between  $-1$  and  $1$  by using the tanh function, why does the hidden state need to use the tanh function again to ensure that the output value range is between  $-1$  and  $1$ ?
5. Implement an LSTM model for time series prediction rather than character sequence prediction.

## 9.3 Deep Recurrent Neural Networks

Up to now, we only discussed RNNs with a single unidirectional hidden layer. In it the specific functional form of how latent variables and observations interact is rather arbitrary. This is not a big problem as long as we have enough flexibility to model different types of interactions. With a single layer, however, this can be quite challenging. In the case of the linear models, we fixed this problem by adding more layers. Within RNNs this is a bit trickier, since we first need to decide how and where to add extra nonlinearity.

In fact, we could stack multiple layers of RNNs on top of each other. This results in a flexible mechanism, due to the combination of several simple layers. In particular, data might be relevant at different levels of the stack. For instance, we might want to keep high-level data about financial market conditions (bear or bull market) available, whereas at a lower level we only record shorter-term temporal dynamics.

Beyond all the above abstract discussion it is probably easiest to understand the family of models we are interested in by reviewing [Fig. 9.3.1](#). It describes a deep RNN with  $L$  hidden layers. Each hidden state is continuously passed to both the next time step of the current layer and the current time step of the next layer.

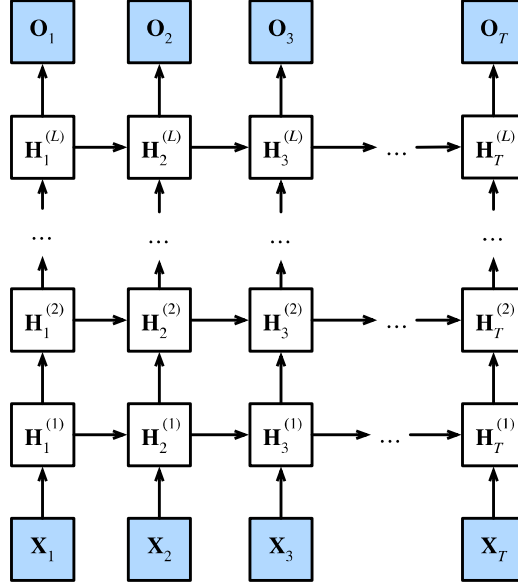


Fig. 9.3.1: Architecture of a deep RNN.

### 9.3.1 Functional Dependencies

We can formalize the functional dependencies within the deep architecture of  $L$  hidden layers depicted in Fig. 9.3.1. Our following discussion focuses primarily on the vanilla RNN model, but it applies to other sequence models, too.

Suppose that we have a minibatch input  $\mathbf{X}_t \in \mathbb{R}^{n \times d}$  (number of examples:  $n$ , number of inputs in each example:  $d$ ) at time step  $t$ . At the same time step, let the hidden state of the  $l^{\text{th}}$  hidden layer ( $l = 1, \dots, L$ ) be  $\mathbf{H}_t^{(l)} \in \mathbb{R}^{n \times h}$  (number of hidden units:  $h$ ) and the output layer variable be  $\mathbf{O}_t \in \mathbb{R}^{n \times q}$  (number of outputs:  $q$ ). Setting  $\mathbf{H}_t^{(0)} = \mathbf{X}_t$ , the hidden state of the  $l^{\text{th}}$  hidden layer that uses the activation function  $\phi_l$  is expressed as follows:

$$\mathbf{H}_t^{(l)} = \phi_l(\mathbf{H}_t^{(l-1)} \mathbf{W}_{xh}^{(l)} + \mathbf{H}_{t-1}^{(l)} \mathbf{W}_{hh}^{(l)} + \mathbf{b}_h^{(l)}), \quad (9.3.1)$$

where the weights  $\mathbf{W}_{xh}^{(l)} \in \mathbb{R}^{h \times d}$  and  $\mathbf{W}_{hh}^{(l)} \in \mathbb{R}^{h \times h}$ , together with the bias  $\mathbf{b}_h^{(l)} \in \mathbb{R}^{1 \times h}$ , are the model parameters of the  $l^{\text{th}}$  hidden layer.

In the end, the calculation of the output layer is only based on the hidden state of the final  $L^{\text{th}}$  hidden layer:

$$\mathbf{O}_t = \mathbf{H}_t^{(L)} \mathbf{W}_{hq} + \mathbf{b}_q, \quad (9.3.2)$$

where the weight  $\mathbf{W}_{hq} \in \mathbb{R}^{h \times q}$  and the bias  $\mathbf{b}_q \in \mathbb{R}^{1 \times q}$  are the model parameters of the output layer.

Just as with MLPs, the number of hidden layers  $L$  and the number of hidden units  $h$  are hyperparameters. In other words, they can be tuned or specified by us. In addition, we can easily get a deep gated RNN by replacing the hidden state computation in (9.3.1) with that from a GRU or an LSTM.

### 9.3.2 Concise Implementation

Fortunately many of the logistical details required to implement multiple layers of an RNN are readily available in high-level APIs. To keep things simple we only illustrate the implementation using such built-in functionalities. Let us take an LSTM model as an example. The code is very similar to the one we used previously in [Section 9.2](#). In fact, the only difference is that we specify the number of layers explicitly rather than picking the default of a single layer. As usual, we begin by loading the dataset.

```
import torch
from torch import nn
from d2l import torch as d2l

batch_size, num_steps = 32, 35
train_iter, vocab = d2l.load_data_time_machine(batch_size, num_steps)
```

The architectural decisions such as choosing hyperparameters are very similar to those of [Section 9.2](#). We pick the same number of inputs and outputs as we have distinct tokens, i.e., `vocab_size`. The number of hidden units is still 256. The only difference is that we now select a nontrivial number of hidden layers by specifying the value of `num_layers`.

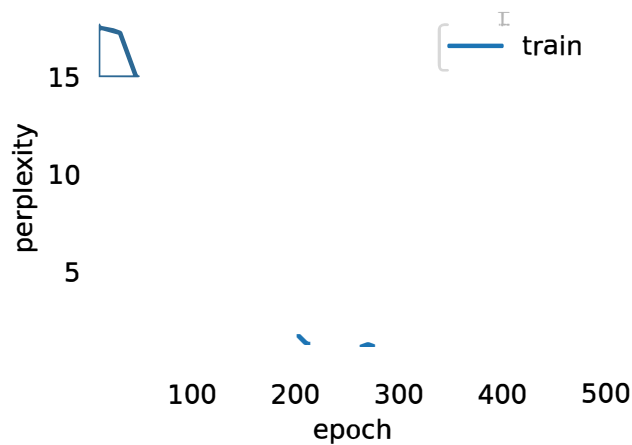
```
vocab_size, num_hiddens, num_layers = len(vocab), 256, 2
num_inputs = vocab_size
device = d2l.try_gpu()
lstm_layer = nn.LSTM(num_inputs, num_hiddens, num_layers)
model = d2l.RNNModel(lstm_layer, len(vocab))
model = model.to(device)
```

### 9.3.3 Training and Prediction

Since now we instantiate two layers with the LSTM model, this rather more complex architecture slows down training considerably.

```
num_epochs, lr = 500, 2
d2l.train_ch8(model, train_iter, vocab, lr, num_epochs, device)
```

```
perplexity 1.0, 179078.9 tokens/sec on cuda:0
time traveller with a slight accession of cheerfulness really thi
travelleryou can show black is white by argument said filby
```



## Summary

- In deep RNNs, the hidden state information is passed to the next time step of the current layer and the current time step of the next layer.
- There exist many different flavors of deep RNNs, such as LSTMs, GRUs, or vanilla RNNs. Conveniently these models are all available as parts of the high-level APIs of deep learning frameworks.
- Initialization of models requires care. Overall, deep RNNs require considerable amount of work (such as learning rate and clipping) to ensure proper convergence.

## Exercises

1. Try to implement a two-layer RNN from scratch using the single layer implementation we discussed in [Section 8.5](#).
2. Replace the LSTM by a GRU and compare the accuracy and training speed.
3. Increase the training data to include multiple books. How low can you go on the perplexity scale?
4. Would you want to combine sources of different authors when modeling text? Why is this a good idea? What could go wrong?

## 9.4 Bidirectional Recurrent Neural Networks

In sequence learning, so far we assumed that our goal is to model the next output given what we have seen so far, e.g., in the context of a time series or in the context of a language model. While this is a typical scenario, it is not the only one we might encounter. To illustrate the issue, consider the following three tasks of filling in the blank in a text sequence:

- I am \_\_\_\_.
- I am \_\_\_\_ hungry.
- I am \_\_\_\_ hungry, and I can eat half a pig.

Depending on the amount of information available, we might fill in the blanks with very different words such as “happy”, “not”, and “very”. Clearly the end of the phrase (if available) conveys significant information about which word to pick. A sequence model that is incapable of taking advantage of this will perform poorly on related tasks. For instance, to do well in named entity recognition (e.g., to recognize whether “Green” refers to “Mr. Green” or to the color) longer-range context is equally vital. To get some inspiration for addressing the problem let us take a detour to probabilistic graphical models.

### 9.4.1 Dynamic Programming in Hidden Markov Models

This subsection serves to illustrate the dynamic programming problem. The specific technical details do not matter for understanding the deep learning models but they help in motivating why one might use deep learning and why one might pick specific architectures.

If we want to solve the problem using probabilistic graphical models we could for instance design a latent variable model as follows. At any time step  $t$ , we assume that there exists some latent variable  $h_t$  that governs our observed emission  $x_t$  via  $P(x_t | h_t)$ . Moreover, any transition  $h_t \rightarrow h_{t+1}$  is given by some state transition probability  $P(h_{t+1} | h_t)$ . This probabilistic graphical model is then a *hidden Markov model* as in Fig. 9.4.1.

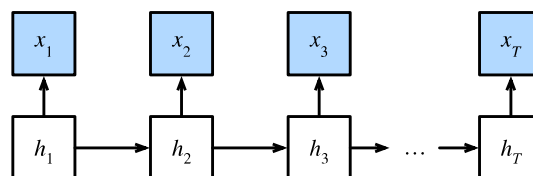


Fig. 9.4.1: A hidden Markov model.

Thus, for a sequence of  $T$  observations we have the following joint probability distribution over the observed and hidden states:

$$P(x_1, \dots, x_T, h_1, \dots, h_T) = \prod_{t=1}^T P(h_t | h_{t-1}) P(x_t | h_t), \text{ where } P(h_1 | h_0) = P(h_1). \quad (9.4.1)$$

Now assume that we observe all  $x_i$  with the exception of some  $x_j$  and it is our goal to compute  $P(x_j | x_{-j})$ , where  $x_{-j} = (x_1, \dots, x_{j-1}, x_{j+1}, \dots, x_T)$ . Since there is no latent variable in  $P(x_j | x_{-j})$ , we consider summing over all the possible combinations of choices for  $h_1, \dots, h_T$ . In case any  $h_i$  can take on  $k$  distinct values (a finite number of states), this means that we need to sum over  $k^T$  terms—usually mission impossible! Fortunately there is an elegant solution for this: *dynamic programming*.

To see how it works, consider summing over latent variables  $h_1, \dots, h_T$  in turn. According to (9.4.1), this yields:

$$\begin{aligned}
& P(x_1, \dots, x_T) \\
&= \sum_{h_1, \dots, h_T} P(x_1, \dots, x_T, h_1, \dots, h_T) \\
&= \sum_{h_1, \dots, h_T} \prod_{t=1}^T P(h_t | h_{t-1}) P(x_t | h_t) \\
&= \sum_{h_2, \dots, h_T} \underbrace{\left[ \sum_{h_1} P(h_1) P(x_1 | h_1) P(h_2 | h_1) \right]}_{\pi_2(h_2) \stackrel{\text{def}}{=}} P(x_2 | h_2) \prod_{t=3}^T P(h_t | h_{t-1}) P(x_t | h_t) \\
&= \sum_{h_3, \dots, h_T} \underbrace{\left[ \sum_{h_2} \pi_2(h_2) P(x_2 | h_2) P(h_3 | h_2) \right]}_{\pi_3(h_3) \stackrel{\text{def}}{=}} P(x_3 | h_3) \prod_{t=4}^T P(h_t | h_{t-1}) P(x_t | h_t) \\
&= \dots \\
&= \sum_{h_T} \pi_T(h_T) P(x_T | h_T).
\end{aligned} \tag{9.4.2}$$

In general we have the *forward recursion* as

$$\pi_{t+1}(h_{t+1}) = \sum_{h_t} \pi_t(h_t) P(x_t | h_t) P(h_{t+1} | h_t). \tag{9.4.3}$$

The recursion is initialized as  $\pi_1(h_1) = P(h_1)$ . In abstract terms this can be written as  $\pi_{t+1} = f(\pi_t, x_t)$ , where  $f$  is some learnable function. This looks very much like the update equation in the latent variable models we discussed so far in the context of RNNs!

Entirely analogously to the forward recursion, we can also sum over the same set of latent variables with a backward recursion. This yields:

$$\begin{aligned}
& P(x_1, \dots, x_T) \\
&= \sum_{h_1, \dots, h_T} P(x_1, \dots, x_T, h_1, \dots, h_T) \\
&= \sum_{h_1, \dots, h_T} \prod_{t=1}^{T-1} P(h_t | h_{t-1}) P(x_t | h_t) \cdot P(h_T | h_{T-1}) P(x_T | h_T) \\
&= \sum_{h_1, \dots, h_{T-1}} \prod_{t=1}^{T-1} P(h_t | h_{t-1}) P(x_t | h_t) \cdot \underbrace{\left[ \sum_{h_T} P(h_T | h_{T-1}) P(x_T | h_T) \right]}_{\rho_{T-1}(h_{T-1}) \stackrel{\text{def}}{=}} \\
&= \sum_{h_1, \dots, h_{T-2}} \prod_{t=1}^{T-2} P(h_t | h_{t-1}) P(x_t | h_t) \cdot \underbrace{\left[ \sum_{h_{T-1}} P(h_{T-1} | h_{T-2}) P(x_{T-1} | h_{T-1}) \rho_{T-1}(h_{T-1}) \right]}_{\rho_{T-2}(h_{T-2}) \stackrel{\text{def}}{=}} \\
&= \dots \\
&= \sum_{h_1} P(h_1) P(x_1 | h_1) \rho_1(h_1).
\end{aligned} \tag{9.4.4}$$

We can thus write the *backward recursion* as

$$\rho_{t-1}(h_{t-1}) = \sum_{h_t} P(h_t | h_{t-1}) P(x_t | h_t) \rho_t(h_t), \quad (9.4.5)$$

with initialization  $\rho_T(h_T) = 1$ . Both the forward and backward recursions allow us to sum over  $T$  latent variables in  $\mathcal{O}(kT)$  (linear) time over all values of  $(h_1, \dots, h_T)$  rather than in exponential time. This is one of the great benefits of the probabilistic inference with graphical models. It is also a very special instance of a general message passing algorithm (Aji & McEliece, 2000). Combining both forward and backward recursions, we are able to compute

$$P(x_j | x_{-j}) \propto \sum_{h_j} \pi_j(h_j) \rho_j(h_j) P(x_j | h_j). \quad (9.4.6)$$

Note that in abstract terms the backward recursion can be written as  $\rho_{t-1} = g(\rho_t, x_t)$ , where  $g$  is a learnable function. Again, this looks very much like an update equation, just running backwards unlike what we have seen so far in RNNs. Indeed, hidden Markov models benefit from knowing future data when it is available. Signal processing scientists distinguish between the two cases of knowing and not knowing future observations as interpolation v.s. extrapolation. See the introductory chapter of the book on sequential Monte Carlo algorithms for more details (Doucet et al., 2001).

### 9.4.2 Bidirectional Model

If we want to have a mechanism in RNNs that offers comparable look-ahead ability as in hidden Markov models, we need to modify the RNN design that we have seen so far. Fortunately, this is easy conceptually. Instead of running an RNN only in the forward mode starting from the first token, we start another one from the last token running from back to front. *Bidirectional RNNs* add a hidden layer that passes information in a backward direction to more flexibly process such information. Fig. 9.4.2 illustrates the architecture of a bidirectional RNN with a single hidden layer.

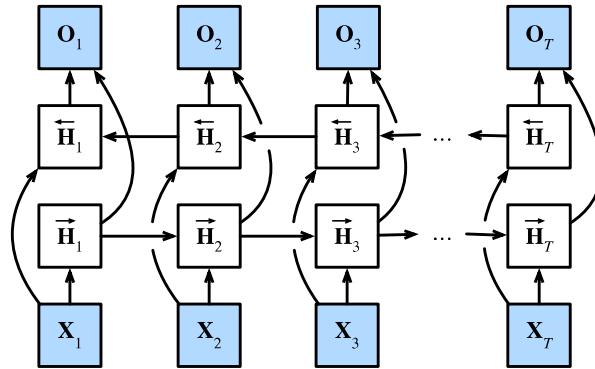


Fig. 9.4.2: Architecture of a bidirectional RNN.

In fact, this is not too dissimilar to the forward and backward recursions in the dynamic programming of hidden Markov models. The main distinction is that in the previous case these equations had a specific statistical meaning. Now they are devoid of such easily accessible interpretations and we can just treat them as generic and learnable functions. This transition epitomizes many of the principles guiding the design of modern deep networks: first, use the type of functional dependencies of classical statistical models, and then parameterize them in a generic form.

## Definition

Bidirectional RNNs were introduced by (Schuster & Paliwal, 1997). For a detailed discussion of the various architectures see also the paper (Graves & Schmidhuber, 2005). Let us look at the specifics of such a network.

For any time step  $t$ , given a minibatch input  $\mathbf{X}_t \in \mathbb{R}^{n \times d}$  (number of examples:  $n$ , number of inputs in each example:  $d$ ) and let the hidden layer activation function be  $\phi$ . In the bidirectional architecture, we assume that the forward and backward hidden states for this time step are  $\vec{\mathbf{H}}_t \in \mathbb{R}^{n \times h}$  and  $\overleftarrow{\mathbf{H}}_t \in \mathbb{R}^{n \times h}$ , respectively, where  $h$  is the number of hidden units. The forward and backward hidden state updates are as follows:

$$\begin{aligned}\vec{\mathbf{H}}_t &= \phi(\mathbf{X}_t \mathbf{W}_{xh}^{(f)} + \vec{\mathbf{H}}_{t-1} \mathbf{W}_{hh}^{(f)} + \mathbf{b}_h^{(f)}), \\ \overleftarrow{\mathbf{H}}_t &= \phi(\mathbf{X}_t \mathbf{W}_{xh}^{(b)} + \overleftarrow{\mathbf{H}}_{t+1} \mathbf{W}_{hh}^{(b)} + \mathbf{b}_h^{(b)}),\end{aligned}\tag{9.4.7}$$

where the weights  $\mathbf{W}_{xh}^{(f)} \in \mathbb{R}^{d \times h}$ ,  $\mathbf{W}_{hh}^{(f)} \in \mathbb{R}^{h \times h}$ ,  $\mathbf{W}_{xh}^{(b)} \in \mathbb{R}^{d \times h}$ , and  $\mathbf{W}_{hh}^{(b)} \in \mathbb{R}^{h \times h}$ , and biases  $\mathbf{b}_h^{(f)} \in \mathbb{R}^{1 \times h}$  and  $\mathbf{b}_h^{(b)} \in \mathbb{R}^{1 \times h}$  are all the model parameters.

Next, we concatenate the forward and backward hidden states  $\vec{\mathbf{H}}_t$  and  $\overleftarrow{\mathbf{H}}_t$  to obtain the hidden state  $\mathbf{H}_t \in \mathbb{R}^{n \times 2h}$  to be fed into the output layer. In deep bidirectional RNNs with multiple hidden layers, such information is passed on as *input* to the next bidirectional layer. Last, the output layer computes the output  $\mathbf{O}_t \in \mathbb{R}^{n \times q}$  (number of outputs:  $q$ ):

$$\mathbf{O}_t = \mathbf{H}_t \mathbf{W}_{hq} + \mathbf{b}_q.\tag{9.4.8}$$

Here, the weight matrix  $\mathbf{W}_{hq} \in \mathbb{R}^{2h \times q}$  and the bias  $\mathbf{b}_q \in \mathbb{R}^{1 \times q}$  are the model parameters of the output layer. In fact, the two directions can have different numbers of hidden units.

## Computational Cost and Applications

One of the key features of a bidirectional RNN is that information from both ends of the sequence is used to estimate the output. That is, we use information from both future and past observations to predict the current one. In the case of next token prediction this is not quite what we want. After all, we do not have the luxury of knowing the next to next token when predicting the next one. Hence, if we were to use a bidirectional RNN naively we would not get a very good accuracy: during training we have past and future data to estimate the present. During test time we only have past data and thus poor accuracy. We will illustrate this in an experiment below.

To add insult to injury, bidirectional RNNs are also exceedingly slow. The main reasons for this are that the forward propagation requires both forward and backward recursions in bidirectional layers and that the backpropagation is dependent on the outcomes of the forward propagation. Hence, gradients will have a very long dependency chain.

In practice bidirectional layers are used very sparingly and only for a narrow set of applications, such as filling in missing words, annotating tokens (e.g., for named entity recognition), and encoding sequences wholesale as a step in a sequence processing pipeline (e.g., for machine translation). In Section 14.7 and Section 15.2, we will introduce how to use bidirectional RNNs to encode text sequences.

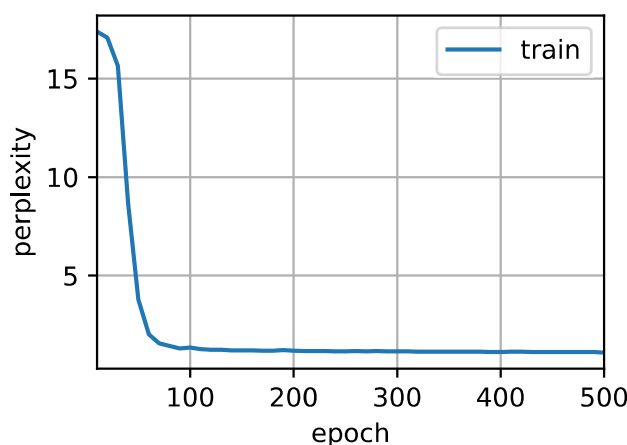
### 9.4.3 Training a Bidirectional RNN for a Wrong Application

If we were to ignore all advice regarding the fact that bidirectional RNNs use past and future data and simply apply it to language models, we will get estimates with acceptable perplexity. Nonetheless, the ability of the model to predict future tokens is severely compromised as the experiment below illustrates. Despite reasonable perplexity, it only generates gibberish even after many iterations. We include the code below as a cautionary example against using them in the wrong context.

```
import torch
from torch import nn
from d2l import torch as d2l

# Load data
batch_size, num_steps, device = 32, 35, d2l.try_gpu()
train_iter, vocab = d2l.load_data_time_machine(batch_size, num_steps)
# Define the bidirectional LSTM model by setting `bidirectional=True`
vocab_size, num_hiddens, num_layers = len(vocab), 256, 2
num_inputs = vocab_size
lstm_layer = nn.LSTM(num_inputs, num_hiddens, num_layers, bidirectional=True)
model = d2l.RNNModel(lstm_layer, len(vocab))
model = model.to(device)
# Train the model
num_epochs, lr = 500, 1
d2l.train_ch8(model, train_iter, vocab, lr, num_epochs, device)
```

```
perplexity 1.1, 119331.5 tokens/sec on cuda:0
time travellerererererererererererererererererererererererererererer
travellerererererererererererererererererererererererererererer
```



The output is clearly unsatisfactory for the reasons described above. For a discussion of more effective uses of bidirectional RNNs, please see the sentiment analysis application in [Section 15.2](#).

## Summary

- In bidirectional RNNs, the hidden state for each time step is simultaneously determined by the data prior to and after the current time step.
- Bidirectional RNNs bear a striking resemblance with the forward-backward algorithm in probabilistic graphical models.
- Bidirectional RNNs are mostly useful for sequence encoding and the estimation of observations given bidirectional context.
- Bidirectional RNNs are very costly to train due to long gradient chains.

## Exercises

1. If the different directions use a different number of hidden units, how will the shape of  $\mathbf{H}_t$  change?
2. Design a bidirectional RNN with multiple hidden layers.
3. Polysemy is common in natural languages. For example, the word “bank” has different meanings in contexts “i went to the bank to deposit cash” and “i went to the bank to sit down”. How can we design a neural network model such that given a context sequence and a word, a vector representation of the word in the context will be returned? What type of neural architectures is preferred for handling polysemy?